



Instituto Politécnico Nacional



Unidad Profesional Interdisciplinaria en Ingeniería
Y Tecnologías Avanzadas

Bases de Datos Distribuidas

Práctica 3 “Particionamiento por año COVID”

Alumnos:

De la Rosa Valdes Itztli Fernanda

Luviano Moreno Alondra

Peña Armas Karina Michelle

Objetivo

Restaurar la base de datos CovidHistorico2, crear filegroups separados por año, definir una función y un esquema de partición, crear una tabla particionada y validar la distribución física y lógica de los datos mediante consultas en SQL Server.

0. Restaurar base de datos desde backup

Se restaura la base de datos a partir del archivo .bak y se indican las ubicaciones de los archivos principal y de log.

```
RESTORE FILELISTONLY
FROM DISK = 'C:\Program Files\Microsoft SQL
Server\MSSQL16.SQLEXPRESS\MSSQL\Backup\covidHistorico2.bak';

RESTORE DATABASE CovidHistorico2
FROM DISK = 'C:\Program Files\Microsoft SQL
Server\MSSQL16.SQLEXPRESS\MSSQL\Backup\covidHistorico2.bak'
WITH
MOVE 'covidHistorico'
TO 'C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\CovidHistorico2.mdf',
MOVE 'covidHistorico_log'
TO 'C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\CovidHistorico2_log.ldf',
REPLACE, RECOVERY;
GO
```

```
/*=====
1) VERIFICAR DISTRIBUCIÓN DE AÑOS EN LA TABLA STAGING
(datoscovid)
=====*/
SELECT YEAR(FECHA_INGRESO) AS Anio, COUNT(*) AS Registros
FROM datoscovid
GROUP BY YEAR(FECHA_INGRESO)
ORDER BY Anio;
GO
```

```
/*=====
2) CREAR FILEGROUPS POR AÑO
=====*/
```

Results Messages

Anio	Registros
2020	3791128
2021	8793503
2022	2680975

1. Verificar distribución de años

Esta consulta permite conocer cuántos registros existen para cada año antes de realizar el particionamiento.

```
SELECT YEAR(FECHA_INGRESO) AS Anio, COUNT(*) AS Registros
FROM datoscovid
GROUP BY YEAR(FECHA_INGRESO)
ORDER BY Anio;
GO
```

2. Crear filegroups por año

Los filegroups separan lógicamente el almacenamiento destinado a cada rango de fechas.

```
ALTER DATABASE CovidHistorico2 ADD FILEGROUP FG_ANTES_2020;
ALTER DATABASE CovidHistorico2 ADD FILEGROUP FG_2020;
ALTER DATABASE CovidHistorico2 ADD FILEGROUP FG_2021;
ALTER DATABASE CovidHistorico2 ADD FILEGROUP FG_2022_MAS;
GO
```

3. Crear archivos físicos (.ndf)

Cada filegroup recibe un archivo físico .ndf donde SQL Server puede almacenar las páginas correspondientes a su partición.

```

ALTER DATABASE CovidHistorico2 ADD FILE
(NAME='FG_ANTES_2020_dat',
 FILENAME='C:\Program Files\Microsoft SQL
Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\FG_ANTES_2020.ndf')
TO FILEGROUP FG_ANTES_2020;
GO

ALTER DATABASE CovidHistorico2 ADD FILE
(NAME='FG_2020_dat',
 FILENAME='C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\FG_2020.ndf')
TO FILEGROUP FG_2020;
GO

ALTER DATABASE CovidHistorico2 ADD FILE
(NAME='FG_2021_dat',
 FILENAME='C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\FG_2021.ndf')
TO FILEGROUP FG_2021;
GO

ALTER DATABASE CovidHistorico2 ADD FILE
(NAME='FG_2022_dat',
 FILENAME='C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\FG_2022.ndf')
TO FILEGROUP FG_2022_MAS;
GO

```

4. Crear función de partición

Con RANGE RIGHT, los valores límite pertenecen a la partición situada a la derecha: P1 contiene fechas anteriores a 2020; P2 el año 2020; P3 el año 2021; y P4 fechas de 2022 en adelante.

```

CREATE PARTITION FUNCTION pf_anio (DATE)
AS RANGE RIGHT FOR VALUES
('2020-01-01','2021-01-01','2022-01-01');
GO

```

5. Crear esquema de partición

El esquema relaciona cada partición definida por pf_anio con el filegroup donde se almacenará.

```
CREATE PARTITION SCHEME ps_anio
AS PARTITION pf_anio
TO (
    FG_ANTES_2020,
    FG_2020,
    FG_2021,
    FG_2022_MAS
);
GO
```

6. Crear tabla particionada

La tabla se crea sobre ps_anio utilizando FECHA_INGRESO como columna de particionamiento. La clave primaria clustered incluye dicha columna.

```
CREATE TABLE covid_particionado (
    Id INT IDENTITY(1,1),
    FECHA_INGRESO DATE NOT NULL,
    ENTIDAD_RES VARCHAR(50),
    EDAD INT,
    CONSTRAINT PK_covid_particionado
        PRIMARY KEY CLUSTERED (FECHA_INGRESO, Id)
)
ON ps_anio(FECHA_INGRESO);
GO
```

7. Insertar muestras de datos

Se insertan muestras de hasta 200000 registros por rango. TRY_CONVERT convierte los valores de texto a DATE o INT y REPLACE elimina comillas almacenadas en los datos.

```

-- Antes de 2020
INSERT INTO covid_particionado (FECHA_INGRESO, ENTIDAD_RES, EDAD)
SELECT TOP (200000)
    TRY_CONVERT(DATE, REPLACE(FECHA_INGRESO, '-', '')),
    REPLACE(ENTIDAD_RES, '-', ''),
    TRY_CONVERT(INT, REPLACE(EDAD, '-', ''))
FROM datoscovid
WHERE TRY_CONVERT(DATE, REPLACE(FECHA_INGRESO, '-', '')) < '2020-01-01';

-- Repetir el mismo SELECT para:
-- 2020: BETWEEN '2020-01-01' AND '2020-12-31'
-- 2021: BETWEEN '2021-01-01' AND '2021-12-31'
-- 2022 en adelante: >= '2022-01-01'
GO

```

Pruebas y validaciones

Prueba 1. Ver número de filas por partición

Permite observar el panorama general de filas registradas en las particiones e índices de la tabla.

```

SELECT p.partition_number AS Particion, p.rows AS Filas
FROM sys.partitions p
JOIN sys.tables t ON p.object_id = t.object_id
WHERE t.name = 'covid_particionado';
GO

```

```

/*=====
PRUEBA 1 - Ver número de filas por partición
Ambas consultas muestran filas por partición,
pero la segunda es más exacta porque se limita al índice clustered.
La primera es útil para ver todo el panorama,
mientras que la segunda es la validación oficial de que los datos
están distribuidos en las particiones correctas.
=====*/
SELECT p.partition_number AS Particion, p.rows AS Filas
FROM sys.partitions p
JOIN sys.tables t ON p.object_id = t.object_id
WHERE t.name = 'covid_particionado';
GO
/*=====

```

5 %

Results Messages

	Particion	Filas
1	1	0
2	2	3991128
3	3	8993503
4	4	2880975

Prueba 2. Validar filas por partición

Limita la revisión al heap o índice clustered, por lo que sirve como validación más directa de la distribución de filas.

```
SELECT p.partition_number, p.rows
FROM sys.partitions p
WHERE p.object_id = OBJECT_ID('covid_particionado')
AND p.index_id IN (0,1);
GO
```

```
/*=====
PRUEBA 2 - Validar filas por partición (índices clustered y heap)
Ambas consultas muestran filas por partición,
pero la segunda es más exacta porque se limita al índice clustered.
La primera es útil para ver todo el panorama,
mientras que la segunda es la validación oficial de que los datos
están distribuidos en las particiones correctas.
=====*/
```

```
SELECT p.partition_number, p.rows
FROM sys.partitions p
WHERE p.object_id = OBJECT_ID('covid_particionado')
AND p.index_id IN (0,1);
GO
```

```
-- PRUEBA 3: Ver en qué FILEGROUP está cada partición
SELECT p.partition_number AS Particion, fg.name AS Filegroup
FROM sys.destination_data_spaces dds
JOIN sys.filegroups fg ON dds.data_space_id = fg.data_space_id
```

partition_number	rows
1	0
2	3991128
3	8993503
4	2880975

Prueba 3. Ver el filegroup de cada partición

Comprueba la asociación entre los números de partición y los filegroups configurados en ps_anio.

```
SELECT p.partition_number AS Particion, fg.name AS Filegroup
FROM sys.destination_data_spaces dds
JOIN sys.filegroups fg ON dds.data_space_id = fg.data_space_id
JOIN sys.partition_schemes ps ON dds.partition_scheme_id = ps.data_space_id
JOIN sys.partitions p ON p.partition_number = dds.destination_id
WHERE ps.name = 'ps_anio';
GO
```

```
-- PRUEBA 3: Ver en qué FILEGROUP está cada partición
SELECT p.partition_number AS Particion, fg.name AS Filegroup
FROM sys.destination_data_spaces dds
JOIN sys.filegroups fg ON dds.data_space_id = fg.data_space_id
JOIN sys.partition_schemes ps ON dds.partition_scheme_id = ps.data_space_id
JOIN sys.partitions p ON p.partition_number = dds.destination_id
WHERE ps.name = 'ps_anio';
GO
```

```
-- PRUEBA 4: Consulta filtrada solo por 2020
```

75 %

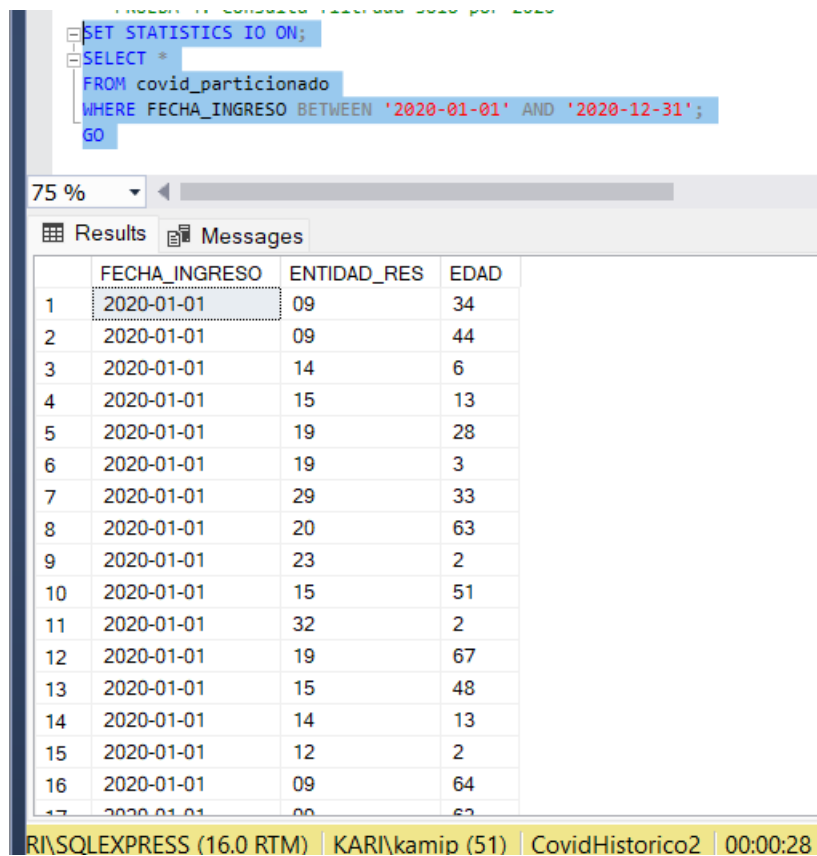
Results Messages

	Particion	Filegroup
177	1	PRIMARY
178	1	PRIMARY
179	1	PRIMARY
180	1	PRIMARY
181	1	PRIMARY
182	1	PRIMARY
183	1	PRIMARY
184	1	PRIMARY
185	1	PRIMARY
186	1	PRIMARY
187	1	PRIMARY
188	1	PRIMARY
189	1	PRIMARY
190	2	PRIMARY
191	3	PRIMARY
192	4	PRIMARY

Prueba 4. Consulta filtrada por 2020

Consulta únicamente los registros de 2020. El filtro sobre la columna de partición permite que SQL Server aproveche el particionamiento.

```
SET STATISTICS IO ON;  
SELECT *  
FROM covid_particionado  
WHERE FECHA_INGRESO BETWEEN '2020-01-01' AND '2020-12-31';  
GO
```



The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays the following query:

```
SET STATISTICS IO ON;  
SELECT *  
FROM covid_particionado  
WHERE FECHA_INGRESO BETWEEN '2020-01-01' AND '2020-12-31';  
GO
```

The bottom pane shows the results of the query in a table with 4 columns: ID, FECHA_INGRESO, ENTIDAD_RES, and EDAD. The table contains 16 rows of data, all for the date 2020-01-01. The status bar at the bottom indicates the server is 'RI\SQLEXPRESS (16.0 RTM)', the user is 'KARI\kamip (51)', the database is 'CovidHistorico2', and the execution time is '00:00:28'.

ID	FECHA_INGRESO	ENTIDAD_RES	EDAD
1	2020-01-01	09	34
2	2020-01-01	09	44
3	2020-01-01	14	6
4	2020-01-01	15	13
5	2020-01-01	19	28
6	2020-01-01	19	3
7	2020-01-01	29	33
8	2020-01-01	20	63
9	2020-01-01	23	2
10	2020-01-01	15	51
11	2020-01-01	32	2
12	2020-01-01	19	67
13	2020-01-01	15	48
14	2020-01-01	14	13
15	2020-01-01	12	2
16	2020-01-01	09	64

Prueba 5. Consulta filtrada por 2021

Consulta únicamente los registros correspondientes al año 2021.

```
SELECT *  
FROM covid_particionado  
WHERE FECHA_INGRESO BETWEEN '2021-01-01' AND '2021-12-31';  
GO
```

The screenshot displays the SQL Server Enterprise Manager interface. At the top, a query window shows the following SQL code:

```
-- PRUEBA 5: Consulta filtrada solo por 2021  
SELECT *  
FROM covid_particionado  
WHERE FECHA_INGRESO BETWEEN '2021-01-01' AND '2021-12-31';  
GO
```

Below the query window, the 'Results' tab is active, showing a table with 4 columns: ID, FECHA_INGRESO, ENTIDAD_RES, and EDAD. The table contains 16 rows of data, all with a FECHA_INGRESO of '2021-01-01'. The status bar at the bottom indicates the connection is to 'KARI\SQLEXPRESS (16.0 RTM)' and the database is 'CovidHistorico2'.

	FECHA_INGRESO	ENTIDAD_RES	EDAD
562	2021-01-01	15	44
563	2021-01-01	14	81
564	2021-01-01	14	62
565	2021-01-01	30	57
566	2021-01-01	26	29
567	2021-01-01	09	68
568	2021-01-01	09	64
569	2021-01-01	22	21
570	2021-01-01	09	17
571	2021-01-01	09	57
572	2021-01-01	15	61
573	2021-01-01	09	21
574	2021-01-01	09	23
575	2021-01-01	08	35
576	2021-01-01	17	54
577	2021-01-01	14	77